

PÓS-GRADUAÇÃO EAD • DISCIPLINA [26E4_2] AULA 02

Processos Agênticos com ChatGPT Work

Configuração de Agentes com AGENTS.md e Arquitetura de Subagentes Especializados

CORPO DOCENTE & ESPECIALIDADE

Prof. Dr. Allan Segovia-Spadini

Doutor em Ciências (USP / TU Delft) • Modelagem Estocástica e Engenharia de Sistemas Agênticos



AGENTS.md

Cadeia Hierárquica GitOps



Subagentes

Concorrência & Isolamento



Sol & Luna

Calibração de Raciocínio



Sandbox Policy

Menor Privilegio & HITL

Roadmap da Aula 2: Trilha Pedagógica

Da Governança de Instruções Hierárquicas à Concorrência de Subagentes Especializados



TRILHA 26E4_2 Estrutura Pedagógica: Situação-Problema do Mundo Real → Solução de Engenharia → Teoria Rigorosa

20 Slides • 5 Laboratórios Interativos

MÓDULO 01 Governança & Cadeia AGENTS.md Anarquia das regras ad-hoc, GitOps determinístico, resolução em 3 camadas (Global → Raiz → Cwd) e precedência semântica top-down. LABORATÓRIO / RIGOR Lab 1: Simulador de Cadeia de Descoberta Slides 03–06 →	MÓDULO 02 Restrições, Truncamento & Fallbacks Teto de 32 KiB (project_doc_max_bytes), política de truncamento, fallbacks legados, isolamento com CODEX_HOME e sintaxe de Code Review Rules. LABORATÓRIO / RIGOR Rigor: Análise de Bytes & Truncamento Slides 07–09 →	MÓDULO 03 Context Rot & Arquitetura de Subagentes Patologias cognitivas de LLMs (Context Pollution & Context Rot), spawning concorrente, barreira wait-for-all e matriz Sol vs Luna com reasoning effort. LABORATÓRIO / RIGOR Lab 2: Simulador de Degradação de Contexto Slides 10–14 →	MÓDULO 04 Custom Agents em TOML & Governança Schema formal de subagentes em TOML, princípio do menor privilégio (read-only vs workspace-write), padrões PR Review e Quiz de Fechamento. LABORATÓRIO / RIGOR Labs 3, 4 e 5: Builder, Runner & Quiz Slides 15–20 →
--	---	--	---

✔ **Competência Final da Aula:**

Projetar monorepositórios com AGENTS.md hierárquico e orquestrar equipes de subagentes concorrentes em TOML com isolamento de privilégios.

MASTERY CHECK

A Anarquia das Instruções Fragmentadas

Situação-Problema do Mundo Real • A Dor Prática das Diretrizes Ad-Hoc e Inconsistentes



 **Situação-Problema do Mundo Real:** O colapso da governança e confiabilidade quando instruções de IA são tratadas como 'prompts soltos' na interface.

FALHA DE ENGENHARIA

CENÁRIO REAL: EQUIPE DE ENGENHARIA SEM AGENTS.MD

Prompts Locais Desconectados

 **Dev Alpha**

"Sempre use pnpm e escreva testes com Jest"

 **Conflito com npm**

 **Dev Beta**

(Sem prompt customizado - usa padrão genérico)

 **Viola linters do CI**

 **Dev Gamma**

"Refatore o módulo de pagamentos rapidamente"

 **Rotação indevida de API Key**



Gargalo Operacional: Retrabalho de 45% do time corrigindo divergências nos Pull Requests

IMPACTOS CRÍTICOS NA INDÚSTRIA

Zero Versionamento (GitOps Nulo)

Prompts residem na memória do navegador. Impossível auditar histórico ou reverter regras que causaram falhas.

Violação de Segurança & Compliance

Regras de segredos e APIs críticas esquecidas, expondo credenciais corporativas a ferramentas públicas.

> Inconsistência em Monorepositórios

Microserviços com pilhas distintas (Python, Go, Node) tratados de forma idêntica e inadequada pelo modelo.

Regra Fundamental: As regras do agente devem residir no repositório, não na cabeça do operador.



✔

Solução de Engenharia: Governança Agêntica como Código (GitOps): O padrão aberto AGENTS.md com compilação determinística no startup.

PADRÃO ABERTO OPENAI

1. ARQUIVOS VERSIONADOS NO GIT

Infraestrutura como Código

repo-root/

AGENTS.md

 ← Diretrizes Globais do Projeto (Lint, PRs)

services/

payments/

AGENTS.override.md

 ← Regras Estritas do Microserviço

index.ts, api.ts, tests/

✔ Versionado no GitHub com PR review, rastreabilidade de autor e histórico de alterações.

2. COMPILAÇÃO DETERMINÍSTICA NO STARTUP

Uma Vez por Sessão

- >_ Todas as Superfícies Sincronizadas

ChatGPT Work Web, Desktop App, Codex CLI e Extensões de IDE (VS Code / JetBrains) carregam exatamente as mesmas regras.
- 🔧 Injeção Automática no Contexto

O motor do Codex concatena as regras antes da primeira interação do modelo, eliminando prompts manuais redundantes.
- ✔ Sem Cache Obsoleto

A cadeia é reconstruída a cada nova execução; editar um arquivo reflete imediatamente na próxima sessão.

Zero intervenção manual: o desenvolvedor apenas entra na pasta e executa o Codex.

A Cadeia de Descoberta e Precedência

Mecânica Algorítmica • Resolução Hierárquica: Global → Repositório → Subdiretórios



 **Mecânica Algorítmica:** Cadeia de Descoberta e Precedência em 3 Camadas: Concatenação Top-Down e Soberania do Nó Folha.

ALGORITMO CODEX

CAMADA 1: GLOBAL SCOPE

~/ .codex/

Avalia o diretório pessoal do desenvolvedor. Aplica preferências persistentes para todos os repositórios da máquina.

ORDEM DE BUSCA (Apenas 1 arquivo):

- 1º AGENTS.override.md
- 2º AGENTS.md

💡 Carrega apenas o primeiro arquivo não-vazio. Pula se ambos estiverem vazios.

CAMADA 2: PROJECT SCOPE

Git Root → CWD

Caminha recursivamente da raiz do Git até o diretório atual de trabalho. Em **cada pasta** do caminho:

POR DIRETÓRIO (Máximo 1 arquivo):

- 1º AGENTS.override.md
- 2º AGENTS.md
- 3º Fallbacks (ex: TEAM_GUIDE.md)

✓ Coleta no máximo 1 arquivo por diretório visitado ao longo do caminho de navegação.

CAMADA 3: MERGE & SOBERANIA

Last-In Wins

Concatenação linear Top-Down unida por linhas em branco. A física do modelo confere precedência ao final do prompt:

Topo: Global ~/ .codex/AGENTS.md



Meio: Raiz repo/AGENTS.md



Fundo: services/payments/AGENTS.override.md (Maior Peso)

Regra de Ouro: Arquivos mais próximos do CWD sobrescrevem diretrizes anteriores.



PARÂMETROS DE EXECUÇÃO Simulador Interativo

DIRETÓRIO ATUAL DE TRABALHO (CWD):

repo-root/ (Raiz do Projeto)

services/payments/ (Microserviço Pagamentos)

services/search/ (Microserviço Busca)

MODIFICADORES DE DESCOBERTA:

- ☐ Criar ~/.codex/AGENTS.override.md
- ☒ Criar payments/AGENTS.override.md
- ☐ Ativar fallback TEAM_GUIDE.md

project_doc_max_bytes:344 / 32768 bytes
(1.05%)

Buffer seguro. Truncamento ocorrerá se exceder 32.768 bytes.

CADEIA DE INSTRUÇÕES COMPILADA (3 ARQUIVOS CARREGADOS)

Ordem de Injeção: Top-Down

- #1 Global (~/.codex) AGENTS.md
- #2 Repo Root (/) AGENTS.md
- #3 services/payments/ AGENTS.override.md **PRIORIDADE MÁXIMA**

```
<!-- Origem: Global (~/.codex) (AGENTS.md) -->
# Global Agreements
- Prefer pnpm when installing dependencies.
- Default linter: ESLint standard.

<!-- Origem: Repo Root (/) (AGENTS.md) -->
# Repository Expectations
- Run `npm run lint` before opening PR.
- Document public utilities in docs/.

<!-- Origem: services/payments/ (AGENTS.override.md) -->
# Payments Service Rules (OVERRIDE)
- Use `make test-payments` instead of `npm test`.
```

✔ **Comportamento Resolvido no CWD:** Comando de teste ativo: `make test-payments` (Regra local sobrescreveu a raiz).

O Limite Físico da Memória de Instrução

Teoria & Formalismo • O Parâmetro `project_doc_max_bytes` e a Saturação de Atenção



 **Teoria & Formalismo Rigoroso:** Restrições Físicas de Memória: O Limite `project_doc_max_bytes` e a Prevenção contra Truncamento Silencioso.

BUFFER 32 KiB

FORMULA DE SATURAÇÃO DE INSTRUÇÃO

$$\sum_{i=1}^k \text{bytes}(F_i) \leq \text{project_doc_max_bytes} = 32.768 \text{ B}$$

O Codex acumula o tamanho em bytes de cada arquivo F_i encontrado no caminho top-down. Se o arquivo $k + 1$ fizer a soma ultrapassar 32 KiB, a concatenação é **interrompida imediatamente**.

Alocação Canônica de Instruções:

Padrão: 32 KiB

Global (~8KB)

Repo Root (~11KB)

Service (~8KB)

Livre

 Configuração customizável em `~/codex/config.toml`: altere para 65536 (64 KiB) apenas se estritamente justificado.

MECÂNICA DE TRUNCAMENTO VS MODULARIZAÇÃO

O Perigo: Arquivo Monolítico Gigante

Se a raiz contiver um `AGENTS.md` de 40 KiB, o Codex truncará o arquivo e descartará TODOS os `AGENTS.md` ou overrides de subdiretórios!

A Solução: Modularização por Diretórios

Mantenha a raiz enxuta (~4 a 8 KiB com regras gerais) e distribua regras de nicho em subpastas. O Codex carrega apenas o que o caminho de execução precisa.

Por que limitar a 32 KiB?

Preservar a densidade de atenção. Instruções prolixas degradam a acurácia do modelo e encarecem cada token gerado.

Boas Práticas: Menos texto, mais diretrizes operacionais e links estritos.

Mapeamento de Fallbacks e Perfis Isolados

Solução de Engenharia • project_doc_fallback_filenames e a Variável CODEX_HOME



⚙️ **Solução de Engenharia:** Configuração Avançada do Codex: Fallbacks para Repositórios Legados e Isolamento com CODEX_HOME.

CONFIG.TOML

1. TRATAMENTO DE ARQUIVOS LEGADOS VIA FALLBACK

```
# ~/.codex/config.toml ou .codex/config.toml
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
```

ORDEM EXATA DE VERIFICAÇÃO EM CADA DIRETÓRIO:

1º

.override.md

2º

AGENTS.md

3º

TEAM_GUIDE.md

4º

.agents.md

✓ O primeiro arquivo existente da lista é adotado; arquivos posteriores no mesmo diretório são ignorados.

2. PERFIS ISOLADOS COM CODEX_HOME

Por padrão, o Codex lê ~/.codex. Ao usar a variável de ambiente, é possível isolar completamente o perfil de execução:

```
$ CODEX_HOME=/opt/ci/.codex \
codex exec "Audit PR rules"
```

Ambiente de CI/CD: Impede que regras locais do desenvolvedor afetem a esteira oficial da empresa.

Auditoria de Conformidade: Garante que robôs de segurança utilizem apenas chaves e regras homologadas.

Isolamento de processos corporativos com zero contaminação de ambiente.

Regras de Code Review em AGENTS.md

Teoria & Formalismo • A Seção Canônica ## Code Review Rules e Boas Práticas



 Teoria & Formalismo: A Seção Canônica ## Code Review Rules em AGENTS.md e a Separação Estrita de Responsabilidades.

GITHUB REVIEW SPEC

ANATOMIA CANÔNICA EM 3 PARTES

Code Review Rules

Experiment cohorts

- **[FLAG]:** Do not filter treatment comparisons on post-exposure behavior, including conversion or retention.

[SAFE PATH]: Build cohorts from assignment or exposure; report conversion as an outcome.

1. TÍTULO

Conciso e focado no domínio

2. SINALIZAÇÃO

O que o modelo deve reprovar

3. CAMINHO SEGURO

Como resolver ou contornar

Posicionamento: Regras gerais no AGENTS.md da raiz; regras específicas no AGENTS.md do microserviço.

DIVISÃO DE RESPONSABILIDADES

>_ Papel do CI/CD Tradicional:

Formatação (Prettier), lint sintático (ESLint), tipos (TypeScript) e execução de suíte de testes. É determinístico, rápido e barato.

🛡️ Papel do Codex Agent Review:

Regras semânticas de negócio, regressão comportamental, brechas de concorrência, autorização e fidelidade à arquitetura corporativa.

⚠️ Não desperdice tokens do LLM pedindo para conferir espaçamento ou ponto-e-vírgula!

Context Pollution e Context Rot

Situação-Problema do Mundo Real • A Degradação da Atenção em Contextos Saturados



⚠ **Situação-Problema do Mundo Real:** As Patologias da Janela de Contexto: Context Pollution (Ruído Intermediário) e Context Rot (Degradação de Atenção).

CHROMA RESEARCH

1. CONTEXT POLLUTION (POLUIÇÃO DE CONTEXTO)

Thread Única

Ao executar tudo em uma única sessão, saídas ruidosas soterram as diretrizes e regras contratuais essenciais:

- ✓ Regras de Negócio Iniciais & Contrato (2.000 tokens)
- ⚠ 1.500 linhas de logs de build do webpack (25.000 tokens)
- ⚠ Leitura integral de 14 arquivos não relacionados (40.000 tokens)
- ⚠ Stack traces duplicados de 40 testes falhando (30.000 tokens)

Resultado: 97% da janela de contexto ocupada por ruído descartável.

2. CONTEXT ROT (PESQUISA EMPÍRICA CHROMA)

Dcaimento da Atenção



- 🗑 **Lost-in-the-Middle:** O modelo prioriza início e fim, ignorando o miolo poluído.
- ⚠ **Alucinações Confiantes:** O modelo preenche lacunas esquecidas com suposições falsas.

A conclusão científica: Manter tudo na mesma thread destrói a confiabilidade do agente.

Decomposição em Subagentes Especializados

Solução de Engenharia • Isolamento de Threads e Resumos Destilados (Summaries)



✓ **Solução de Engenharia:** Decomposição de Tarefas em Subagentes Especializados com Retorno de Resumos Destilados.

DESACOPLAMENTO CONCORRENTE

FLUXO DE EXECUÇÃO COM BARRAMENTO DE CONTEXTO LIMPO

Zero Context Pollution

Main Agent (Orquestrador)
Thread Principal • Contexto Limpo

- Requisitos e restrições do contrato
- Julgamento de negócios e decisões
- Síntese e entrega do entregável final

✓ **Fidelidade de Atenção Preservada (> 98%)**

Spawn



Summary

pr_explorer

Lê 42 arquivos e faz AST diff local (35k tokens na thread isolada)

Retorna: Resumo de 200 tokens com rotas alteradas

reviewer (Security)

Executa simulação de ataque e race conditions (28k tokens na thread)

Retorna: 1 vulnerabilidade em auth.ts:42

docs_researcher

Consulta servidor MCP de documentação da OpenAI (15k tokens)

Retorna: Confirmação de compatibilidade v2

Regra Prática de Engenharia:

Use paralelismo agressivo para tarefas **read-heavy** (exploração, testes, logs, auditoria). Em fluxos **write-heavy**, evite edições simultâneas nos mesmos arquivos para prevenir colisões de estado.



SIMULADOR DE CARGA COGNITIVA

Context Rot Lab

Avance o fluxo de depuração para observar a saturação da janela de contexto e a degradação dos pesos de atenção no LLM.

ETAPA DA INVESTIGAÇÃO:

1. Leitura de Contrato & Setup

2. Exploração de 60 Arquivos

3. Execução de Suíte de Testes (120 testes)

4. Logs de Build & Webpack Bundler

DADOS INSERIDOS NA ETAPA:

Carregado AGENTS.md e contrato de serviço.

Reiniciar Experimento

SINGLE-AGENT (MONOLÍTICO)



Volume na Janela Principal:

2.500 tokens

Fidelidade de Atenção (Chroma Index):

98%

Início de poluição: instruções originais ainda recuperáveis com degradação leve.

MULTI-SUBAGENT (DESACOPLADO)



Volume na Janela Principal:

1.200 tokens

Fidelidade de Atenção (Chroma Index):

99%

✓ Contexto principal blindado. Subagentes absorveram 88.900 tokens em threads descartáveis e retornaram apenas síntese destilada.

Economia de Contexto: 96,6% menos tokens na thread principal • Custo operacional menor e raciocínio 100% ancorado.

Orquestração, Ciclo de Vida e Sincronização

Teoria & Formalismo • Spawning, Threads Concorrentes e Barreira Wait-for-All



 **Teoria & Formalismo:** O Protocolo de Orquestração: Spawning, Concorrência, Barreira Wait-for-All e Consolidação Final.

WAIT-FOR-ALL BARRIER

FASE 01

Disparo

Spawning & Despacho

O orquestrador identifica tarefas independentes e instancia subagentes em threads dedicadas.

Triggers oficiais:

- Prompt: "Spawn one agent per point"
- AGENTS.md / SKILL.md
- Inteligência Proativa (Ultra)

Alocação de PID / Thread ID

FASE 02

Paralelo

Execução Concorrente

Subagentes executam simultaneamente em sandboxes isoladas, consumindo ferramentas independentes.

Controle de Concorrência:

`agents.max_concurrent_threads_per_session`
(Padrão: 6 a 8 threads simultâneas)

Inspeção no CLI via `/agent`

FASE 03

Barreira

Barreira Wait-for-All

O orquestrador suspende a emissão de respostas até que TODAS as threads conclua suas avaliações.

Sincronização Estrita:

Impede respostas fragmentadas ou alucinações baseadas em dados parciais não homologados.

Join Barrier Síncrono

FASE 04

Consolidação

Destilação & Entrega

A thread principal unifica os resumos destilados, categoriza os achados e apresenta o resultado limpo.

Artefato Consolidado:

- Tabela com severidades
- Links diretos de arquivo e linha
- Sugestão unificada de ação

Fechamento de Threads

Controles de Inspeção por Superfície:

CLI: `/agent` alterna entre threads • **Desktop App:** Painel lateral Subagents • **IDE:** Background-Agent Panel com Stop / Open.

Matriz de Modelos e Esforço de Raciocínio

Teoria & Formalismo • Calibração de Inteligência, Latência e Custo de Tokens



 **Teoria & Formalismo:** Matriz de Seleção de Modelos e Calibração de Esforço de Raciocínio (model_reasoning_effort).

SOL vs LUNA

1. SELEÇÃO CANÔNICA DE MODELO POR TIPO DE CARGA

 **gpt-6.1-sol**
Máxima Capacidade Cognitiva

- Tarefas ambíguas e multi-etapas
- Revisão profunda de segurança
- Planejamento e refatorações amplas
- Diagnóstico de race conditions

Perfil: Reviewer, Security, Architect

 **gpt-6-luna**
Velocidade & Baixo Custo

- Varredura rápida de diretórios
- Mapeamento de símbolos e AST
- Verificação de APIs em docs
- Fixes cirúrgicos triviais

Perfil: Explorer, Docs, UI Fixer

2. ESPECTRO DE ESFORÇO DE RACIOCÍNIO

low / medium

Velocidade Máxima

Ideal para tarefas diretas, pesquisas sintáticas e leitura de documentação.

high

Análise de Casos de Borda

Obrigatório para revisão de PRs críticos, refatorações e testes de regressão.

xhigh / max / ultra

Raciocínio Profundo & Proatividade

Permite delegação proativa no ChatGPT Work Web sem necessidade de pedir no prompt.

Hierarquia de Resolução: Prompt explícito → Custom Agent TOML → [agents] default → Herança do pai.

✓ Calibre Sol + High para auditores; Luna + Low para varredores.



 **Teoria & Formalismo:** Anatomia Declarativa de Custom Agents: O Schema Oficial TOML em `.codex/agents/*.toml`.

SCHEMA TOML

`.CODEX/AGENTS/REVIEWER.TOML`

Escopo de Projeto

```
# 1. Campos Canônicos Obrigatórios
name = "reviewer"
description = "PR reviewer focused on correctness and security."

# 2. Configurações de Modelo e Isolamento
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"

# 3. Instruções de Comportamento
developer_instructions = """
Review code like an owner.
Prioritize correctness, security and missing tests.
Lead with concrete findings and reproduction steps.
"""
```

✓ O campo `name` é a fonte da verdade para o Codex instanciar o agente.

ESPECIFICAÇÃO FORMAL DOS CAMPOS

name (Obrigatório) String
Identificador usado no prompt ou em comandos de delegação.

description (Obrigatório) String
Guia para o LLM orquestrador saber quando delegar para este agente.

developer_instructions (Obrigatório) String
Instruções nucleares que governam o comportamento e tom do subagente.

sandbox_mode (Opcional) read-only | write
Sobrescreve a política de isolamento do turno pai.

Localizações: `~/ .codex/agents/` (pessoal) ou `.codex/agents/` (projeto).

🛡️ **Governança Corporativa:** Políticas de Sandbox, Princípio do Menor Privilégio e Portões HITL em Subagentes Concorrentes.

LEAST PRIVILEGE

1. PRINCÍPIO DO MENOR PRIVILÉGIO EM SANDBOXES

🔒 **sandbox_mode = "read-only"**

Obrigatório para `explorer`, `reviewer` e `docs_researcher`. O subagente pode ler código e executar comandos de verificação, mas tem o sistema de arquivos bloqueado para alterações.

>_ **sandbox_mode = "workspace-write"**

Restrito exclusivamente a agentes de correção e implementação (ex: `ui_fixer`, `worker`), apenas após a causa raiz ter sido comprovada por subagentes de leitura.

Regra: Nunca conceda permissão de escrita a agentes cujo propósito seja apenas auditoria ou busca.

2. INTERRUPTÕES E MODOS DE APROVAÇÃO

>_ **Sessão Interativa no CLI**

Se uma thread secundária solicitar uma ação restrita, o CLI sobrepõe o pedido na tela principal. O operador pressiona a tecla `o` para abrir e auditar a thread do subagente antes de responder.

⚠️ **Falha Segura em Automações (Fail-Safe)**

Em esteiras de CI/CD ou fluxos headless onde o operador não pode ser consultado, qualquer tentativa de ação que demande nova aprovação **falha imediatamente** e reporta o erro ao orquestrador.

✓ **Princípio da Reversibilidade:** Governança rígida garantindo que agentes não ajam no escuro.

Laboratório Interativo: Custom Agent Builder

Prática Aplicada • Interface Interativa de Criação e Validação de TOML



CONFIGURADOR DECLARATIVO

Custom Agent TOML

reviewer

pr_explorer

docs_researcher

NAME (OBRIGATÓRIO):

reviewer

DESCRIPTION (PARA O ORQUESTRADOR):

PR reviewer focused on correctness, security, and missing tests.

MODELO:

gpt-6.1-sol

REASONING EFFORT:

medium

POLÍTICA DE SANDBOX:

read-only (Bloqueio total de escrita)

DEVELOPER_INSTRUCTIONS:

Review code like an owner.
Prioritize correctness, security, behavior regressions, and missing test coverage.
Lead with concrete findings, include reproduction steps when possible, and avoid style-only comments unless they hide a real bug.

✓ Schema Oficial Válido

.codex/agents/reviewer.toml

ARQUIVO TOML COMPILADO

Copiar TOML

```
name = "reviewer"
description = "PR reviewer focused on correctness, security, and missing tests."
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"

developer_instructions = """
Review code like an owner.
Prioritize correctness, security, behavior regressions, and missing test coverage.
Lead with concrete findings, include reproduction steps when possible, and avoid style-only comments unless
they hide a real bug.
"""
```

✓ Salve este arquivo em `.codex/agents/reviewer.toml`. O Codex o carregará automaticamente como camada de sessão isolada.



PADRÃO 1: TRIPLO PR REVIEW

Read-Only Audit

Auditoria de alta precisão em Pull Requests corporativos decomposta em 3 papéis não conflitantes:

- pr_explorer (Luna • High • Read-Only)**

Varre o repositório, mapeia arquivos afetados e coleta evidências sem propor correções.
- reviewer (Sol • Medium • Read-Only)**

Focado em riscos reais, segurança, regressões de comportamento e falta de testes.
- docs_researcher (Luna • High • MCP Server)**

Consulta o servidor MCP da documentação oficial para conferir versões de APIs e parâmetros.

Prompt: "Review this branch against main using pr_explorer, reviewer and docs_researcher."

PADRÃO 2: FRONTEND DEBUGGING

Evidence → Fix

Resolução de falhas de interface e testes ponta-a-ponta com captura real de evidências:

- browser_debugger (Sol • Chrome DevTools MCP)**

Abre o navegador headless, reproduz o erro, captura screenshots e logs de rede/console.
- code_mapper (Luna • High • Read-Only)**

Rastreia o componente React e a rota de API responsável pela falha comprovada.
- ui_fixer (Luna • Workspace-Write)**

Apenas após a evidência ser isolada, edita o arquivo alvo com a menor alteração defensável.

Princípio da Reversibilidade: Só altera código quando a falha estiver reproduzida com provas.

Laboratório Interativo: PR Review Runner

Prática Aplicada • Simulação de Spawning, Concorrência e Barreira de Sincronização



ORQUESTADOR CONCORRENTE

Pronto

Dispare o padrão Triplo PR Review para acompanhar o spawning simultâneo, avanço das threads e a barreira de sincronização.

▶ Disparar Orquestração

↺

1. pr_explorer (Luna)

0%

2. reviewer (Sol)

0%

3. docs_researcher (MCP)

0%

Barreira ativa: Resposta ao usuário só após conclusão total.

PARECER CONSOLIDADO DO PR (BRANCH FEATURE/AUTH VS MAIN)

Soberania do Agente Principal



Clique em "Disparar Orquestração" para iniciar o review multi-agente.

Veredito do Orquestrador:

Aguardando execução dos subagentes.

Quiz Interativo de Fixação

Avaliação e Fechamento • 5 Questões Avançadas sobre AGENTS.md e Subagentes



❓ QUESTÃO 1 DE 5

Aula 2: AGENTS.md & Subagentes

Como o Codex resolve conflitos de regras quando existem arquivos de instruções no escopo global (~/.codex), na raiz do repositório e em um subdiretório local?

- A) O arquivo global substitui silenciosamente todos os arquivos do repositório, descartando regras locais.
- B) O Codex concatena os arquivos de cima para baixo (top-down), de modo que o arquivo mais próximo do diretório atual de trabalho (CWD) aparece por último no prompt combinado e tem precedência semântica.
- C) O modelo faz uma média ponderada dos tokens e sorteia aleatoriamente qual linter utilizar.
- D) Apenas arquivos com o nome AGENTS.override.md são carregados; todos os demais são ignorados por padrão.

DESEMPENHO & FIXAÇÃO

🏆 0 / 5 Acertos

Selecione uma alternativa para inspecionar o gabarito comentado e a fundamentação formal de engenharia.

CHECKLIST DE DOMÍNIO DA AULA 2:

- ✓ Cadeia de precedência Top-Down (CWD wins)
- ✓ Limite de 32 KiB (project_doc_max_bytes)
- ✓ Isolamento de ruído com subagentes (Chroma)
- ✓ TOML Schema oficial de Custom Agents
- ✓ Sandbox Least Privilege em read-only